

Technical Test: AI Automation Engineer

Context

You're joining a team that has built an AI-powered translation solution. Your job is to extend it to new teams and possible file formats. For this test, you'll build a small document translation pipeline that extracts content from business files, passes it through a translation service, and writes the result back.

This reflects the daily work of the role: parsing real-world documents, integrating with APIs, and delivering usable output to non-technical teams.

Format

Take-home test. One week limit; spend what you think is reasonable.

The Task

Create mock input files in Excel, PDF, and/or Word format containing French text (e.g., product descriptions, internal memos, or similar business content).

Step 1: Extract

Parse each file and extract the translatable text content.

Step 2: Translate

Send the extracted text to the translation service below. **Do not call a real LLM.** Use this mock as-is:

```
def translate(text: str, source: str = "fr", target: str = "en") → str:
    """Mock translation service. Replace with real API later."""
    return f"[EN] {text}"
```

Step 3: Write back

Write the translated content back to new files:

- **Excel:** Add a `description_en` column next to the original. Keep all other columns intact.
- **PDF:** Generate a new PDF with the translated text. Basic formatting is acceptable.
- **Word:** Replace the French text with the English translation. **Preserve the original formatting** (bold, bullets, table structure).

Bonus Points

- **Power Automate flow:** Design or build a flow where a user uploads a file in Teams, the flow calls your endpoint, and returns the translated file. A working export, annotated screenshots, a detailed written description, or a Mermaid diagram are all acceptable.
- **Lambda handler:** Write a `handler.py` structured as an AWS Lambda function. It should accept a file event, run the translation pipeline, and write the result back (e.g., to S3). Include a sample event payload for local testing. **Do not actually deploy anything to AWS.** We just want to see how you'd structure the code.
- **Tests:** Unit tests, integration tests, or a test script demonstrating the pipeline works.
- **AI tool configuration:** Show how you organize your AI setup: skills, hooks, steering files, agents, rules, etc.
- **Anything else:** If you see something that would make this more production-ready, do it.

AI Usage

Use AI tools freely. Cursor, Kiro, Claude Code, GitHub Copilot, ChatGPT, whatever is part of your workflow.

Include your AI configuration in the repo:

- `.kiro/` folder (steering files, hooks, agents)
- `.cursor/` folder (rules, settings)
- `.claude/` or `CLAUDE.md`
- Any `.agents/`, `.github/copilot-instructions.md`, or equivalent

We want to see how you set up and guide your AI tools, not just what they produce.

Deliverables

A **git repository** containing:

<code>README.md</code>	<code># Setup, usage, design decisions, known limitations</code>
<code>src/</code>	<code># Your source code</code>
<code>input/</code>	<code># Mock source files (Excel, PDF, Word)</code>
<code>output/</code>	<code># Generated translated files (Excel, PDF, Word)</code>
<code>requirements.txt</code>	<code># Or pyproject.toml</code>
<code>.kiro/</code>	<code># (if applicable)</code>
<code>.cursor/</code>	<code># (if applicable)</code>
<code>...</code>	<code># Any other config</code>

The README should cover:

1. How to install and run the pipeline
2. Libraries used and why
3. What you'd change if the mock became a real API with rate limits and 50,000 rows
4. Known limitations or shortcuts taken
5. AI tools used and how they helped

Rules

- Use any libraries and AI tools you want.
- Do not call external LLM APIs for translation. Use only the provided mock.
- Do not deploy anything to AWS or any cloud provider. Everything should run locally.
- Commit history is appreciated. We'd rather see a natural progression than one squashed commit.

Follow-up

After submission, we'll schedule a short live debrief where you walk us through your choices.

Questions?

If anything is unclear, ask. In the real job, you'd do the same before building.